

LAPORAN TUGAS BESAR

Mata Kuliah Administrasi Basis Data

Program Studi Sistem Informasi

IMPLEMENTASI DAN ANALISIS PERFORMA QUERY POSTGRESQL PADA SISTEM INFORMASI HIDDEN KOS

Institut Teknologi Kalimantan — Balikpapan, 2026

Identitas Kelompok

Keterangan	Isi
Mata Kuliah	Administrasi Basis Data
Program Studi	Sistem Informasi
Semester / Tahun	Genap / 2025–2026
Dosen Pengampu	[Nama Dosen]

Anggota Kelompok

No	NIM	Nama Lengkap
1	10241034	Muchammad Maulana
2	10241061	Patra Ananda
3	10241070	Tika Mila Wahyuni
4	10251118	Vanessa Marie Tandiarru

Daftar Isi

- 1. [Pendahuluan](#)
 - 2. [Deskripsi Sistem](#)
 - 3. [Perancangan Basis Data \(ERD\)](#)
 - 4. [Implementasi PostgreSQL](#)
 - 5. [Strategi Penggunaan Indeks](#)
 - 6. [Analisis Query dengan EXPLAIN](#)
 - 7. [Kesimpulan](#)
 - 8. [Referensi](#)
 - 9. [Lampiran](#)
-

1. Pendahuluan

1.1 Latar Belakang

Pengelolaan kos-kosan merupakan salah satu bisnis properti yang berkembang pesat, khususnya di kota-kota pelajar dan industri seperti Balikpapan. Namun, mayoritas pengelolaan kos masih dilakukan secara manual atau menggunakan sistem yang tidak terintegrasi, sehingga menimbulkan berbagai permasalahan seperti kesulitan memantau ketersediaan kamar, pencatatan pembayaran yang tidak terstruktur, hingga lambatnya penanganan permintaan maintenance.

Sistem Informasi Hidden Kos dirancang untuk menjawab permasalahan tersebut dengan memanfaatkan basis data relasional PostgreSQL sebagai fondasi penyimpanan dan pengolahan data. Dalam sistem yang menangani data dalam jumlah besar — ratusan kamar, ribuan transaksi sewa, dan puluhan ribu record pembayaran — optimasi akses data menjadi krusial. Tanpa strategi yang tepat, setiap query ke database akan melakukan Sequential Scan (membaca seluruh baris tabel satu per satu), yang berdampak langsung pada lambatnya respons sistem.

Penggunaan index pada PostgreSQL merupakan solusi utama untuk mengatasi masalah ini. Index memungkinkan database engine melakukan Index Scan — langsung melompat ke baris yang relevan tanpa membaca seluruh tabel — sehingga execution time dapat berkurang secara signifikan. Proyek ini mengimplementasikan dan menganalisis efektivitas berbagai strategi index pada sistem Hidden Kos menggunakan `EXPLAIN ANALYZE` dan divisualisasikan melalui dashboard Streamlit.

1.2 Rumusan Masalah

1. Bagaimana merancang struktur basis data yang optimal untuk sistem informasi manajemen kos-kosan?
2. Bagaimana mengimplementasikan index pada PostgreSQL untuk tabel-tabel utama sistem Hidden Kos?
3. Bagaimana memastikan query yang dijalankan menggunakan Index Scan dibandingkan Sequential Scan pada kondisi yang tepat?
4. Bagaimana hasil analisis `EXPLAIN ANALYZE` menunjukkan perbedaan performa query sebelum dan sesudah implementasi index?

1.3 Tujuan

- Merancang ERD yang merepresentasikan kebutuhan data Sistem Informasi Hidden Kos dengan 11 entitas.
- Mengimplementasikan skema basis data pada PostgreSQL dengan data dummy berskala besar (>50.000 record tabel sewa).
- Menerapkan index yang sesuai berdasarkan pola akses data nyata dari aplikasi Streamlit.
- Menganalisis dan membandingkan performa query menggunakan `EXPLAIN ANALYZE` dalam kondisi dengan dan tanpa index.
- Memvisualisasikan hasil analisis performa melalui dashboard Streamlit interaktif.

1.4 Batasan Proyek

- Menggunakan PostgreSQL sebagai database management system utama.
- Data dummy minimal 1.000 record per tabel utama (Sewa, Pembayaran, Penyewa), dengan tabel Sewa mencapai >50.000 record untuk pengujian performa yang representatif.

- Fokus pada query SELECT (pencarian kamar, riwayat sewa, dan riwayat pembayaran); operasi INSERT/UPDATE/DELETE tidak dianalisis performanya.
- Analisis performa dilakukan menggunakan `EXPLAIN ANALYZE` dengan output berupa execution plan PostgreSQL.
- Visualisasi hasil menggunakan Streamlit yang dijalankan secara lokal.
- Studi kasus berfokus pada tiga skenario utama: riwayat sewa per penyewa, pencarian nama penyewa, dan riwayat sewa per kamar.

2. Deskripsi Sistem

2.1 Gambaran Umum Sistem

Sistem Informasi Hidden Kos adalah aplikasi manajemen kos-kosan berbasis web yang dibangun menggunakan Streamlit (Python) dengan PostgreSQL sebagai backend database. Sistem ini melayani dua kelompok pengguna utama:

- **Pemilik Kos (Admin):** dapat memantau seluruh data kamar, pendapatan, pembayaran, data penyewa aktif, dan mengelola permintaan maintenance.
- **Penyewa:** dapat melihat kamar yang tersedia, memantau status sewa aktif dan jatuh tempo pembayaran, melihat riwayat pembayaran, dan mengajukan permintaan maintenance.

Sistem mengelola dua properti kos: **Hidden Kost Origin** dan **Hidden Kost Lux**, masing-masing dengan 150 kamar yang terbagi dalam 3 lantai dan 3 tipe kamar (Tipe A, B, C) dengan harga sewa berbeda.

Alur kerja umum sistem:

1. Penyewa mendaftar dan melengkapi profil (pekerjaan, instansi, dokumen KTP).
2. Admin menyetujui dan mengaktifkan akun penyewa.
3. Penyewa memilih kamar kosong dan melakukan transaksi sewa.
4. Sistem mencatat pembayaran per periode (bulanan) dan memantau jatuh tempo.
5. Penyewa dapat mengajukan maintenance; admin mengelola status penanganan.

2.2 Kebutuhan Data (Requirement Analysis)

Entitas yang diidentifikasi:

Entitas	Deskripsi	Atribut Utama
roles	Peran pengguna sistem	id_roles, nama_role
users	Akun pengguna (login)	id_user, email, password, id_roles
profil_penyewa	Data lengkap penyewa	id_penyewa, nama_lengkap, pekerjaan, instansi, status, id_user
kos	Data properti kos	id_kos, nama_kos, alamat
tipe_kamar	Kategori dan harga kamar	id_tipe_kamar, kategori, deskripsi_fasilitas, harga_sewa

Entitas	Deskripsi	Atribut Utama
kamar	Data setiap unit kamar	id_kamar, nomor, luas, lantai, status, id_kos, id_tipe_kamar
sewa	Transaksi sewa kamar	id_sewa, tanggal_mulai, tanggal_akhir, id_penyewa, id_kamar
pembayaran	Data pembayaran sewa	id_pembayaran, nominal, metode_bayar, id_sewa
periode_pembayaran	Rincian per periode bulanan	id_periode, periode_bayar, status, id_pembayaran
request_maintenance	Pengajuan perbaikan kamar	id_request_maintenance, deskripsi, id_penyewa
riwayat_maintenance	Riwayat penanganan maintenance	id_riwayat_maintenance, tanggal_mulai, tanggal_selesai, status, id_request_maintenance

Relasi antar entitas:

Relasi	Entitas A	Entitas B	Kardinalitas
Memiliki role	roles	users	1:N
Memiliki profil	users	profil_penyewa	1:1 (opsional)
Mengelola	kos	kamar	1:N
Bertipe	tipe_kamar	kamar	1:N
Menyewa	profil_penyewa	sewa	1:N (opsional)
Disewa pada	kamar	sewa	1:N (opsional)
Membayar	sewa	pembayaran	1:N
Dirinci per periode	pembayaran	periode_pembayaran	1:N
Mengajukan	profil_penyewa	request_maintenance	1:N (opsional)
Ditangani pada	request_maintenance	riwayat_maintenance	1:1 (opsional)

3. Perancangan Basis Data (ERD)

3.1 Entity Relationship Diagram (ERD)

🔗 [ARAHAN]: Sisipkan gambar ERD dari dbdiagram.io di sini (screenshot dari laporan PDF halaman 7).

[Sisipkan gambar ERD di sini]

Gambar 3.1 — Entity Relationship Diagram Sistem Informasi Hidden Kos (sumber: dbdiagram.io)

3.2 Penjelasan ERD

Entitas dan Atribut:

- **roles**: primary key `id_roles`, atribut `nama_role` (UNIQUE). Menyimpan dua nilai: `admin` dan `penyewa`.
- **users**: primary key `id_user`, atribut `email` (UNIQUE), `password`, foreign key `id_roles` → **roles**.
- **profil_penyewa**: primary key `id_penyewa`, atribut `nama_lengkap`, `pekerjaan`, `instansi`, `deskripsi_pekerjaan`, `status` (ENUM: Aktif/Tidak aktif), `url_ktp`, foreign key `id_user` (UNIQUE) → **users**.
- **kos**: primary key `id_kos`, atribut `nama_kos` (UNIQUE), `alamat`.
- **tipe_kamar**: primary key `id_tipe_kamar`, atribut `kategori` (UNIQUE), `deskripsi_fasilitas`, `harga_sewa`.
- **kamar**: primary key `id_kamar`, atribut `nomor`, `luas`, `lantai`, `status` (ENUM: Sedang disewa/Kosong), foreign key `id_kos` → **kos**, `id_tipe_kamar` → **tipe_kamar**.
- **sewa**: primary key `id_sewa`, atribut `tanggal_mulai`, `tanggal_akhir`, foreign key `id_penyewa` → **profil_penyewa**, `id_kamar` → **kamar**.
- **pembayaran**: primary key `id_pembayaran`, atribut `nominal`, `metode_bayar` (ENUM: Tunai/Transfer), foreign key `id_sewa` → **sewa**.
- **periode_pembayaran**: primary key `id_periode`, atribut `periode_bayar` (format YYYY-MM), `status` (ENUM: Berhasil/Gagal), foreign key `id_pembayaran` → **pembayaran**.
- **request_maintenance**: primary key `id_request_maintenance`, atribut `deskripsi`, foreign key `id_penyewa` → **profil_penyewa**.
- **riwayat_maintenance**: primary key `id_riwayat_maintenance`, atribut `tanggal_mulai`, `tanggal_selesai`, `status` (ENUM: Tertunda/Sedang dikerjakan/Selesai), foreign key `id_request_maintenance` (UNIQUE) → **request_maintenance**.

Relasi:

- **roles** — **users**: relasi **one-to-many**, karena satu role (misal "penyewa") dapat dimiliki banyak akun, namun setiap akun hanya memiliki satu role.
- **users** — **profil_penyewa**: relasi **one-to-one opsional**, karena akun admin tidak memerlukan profil penyewa; hanya akun dengan role penyewa yang memiliki profil.
- **profil_penyewa** — **sewa**: relasi **one-to-many opsional**, karena satu penyewa dapat melakukan beberapa transaksi sewa di waktu berbeda, dan penyewa baru mungkin belum pernah menyewa.
- **kamar** — **sewa**: relasi **one-to-many opsional**, karena satu kamar dapat tercatat pada beberapa sewa di periode berbeda.
- **kos** — **kamar**: relasi **one-to-many**, karena satu kos memiliki banyak kamar namun setiap kamar hanya berada pada satu kos.
- **tipe_kamar** — **kamar**: relasi **one-to-many**, karena satu tipe kamar dapat digunakan oleh banyak kamar dengan fasilitas dan harga yang sama.
- **sewa** — **pembayaran**: relasi **one-to-many**, karena satu sewa dapat memiliki beberapa transaksi pembayaran (misal dibayar bertahap per semester).
- **pembayaran** — **periode_pembayaran**: relasi **one-to-many**, karena satu pembayaran dapat mencakup beberapa periode bulanan.
- **profil_penyewa** — **request_maintenance**: relasi **one-to-many opsional**, karena tidak semua penyewa mengajukan maintenance.

- `request_maintenance` — `riwayat_maintenance`: relasi **one-to-one opsional**, karena setiap request hanya memiliki satu riwayat penanganan, dan request baru belum tentu sudah ditangani.

3.3 Normalisasi

Skema basis data Sistem Informasi Hidden Kos telah memenuhi **Third Normal Form (3NF)** karena:

- Setiap tabel memiliki primary key yang jelas dan atomik (`SERIAL` / integer).
- Tidak terdapat dependensi parsial — setiap atribut non-key bergantung penuh pada primary key tabelnya (sesuai 2NF). Contoh: `harga_sewa` disimpan di `tipe_kamar`, bukan di `kamar`, sehingga tidak ada redundansi data harga.
- Tidak terdapat dependensi transitif (sesuai 3NF). Contoh: data profil penyewa dipisahkan dari tabel `users` sehingga atribut seperti `pekerjaan` dan `instansi` hanya bergantung pada `id_penyewa`, bukan pada `id_user`.
- Penggunaan tipe ENUM membatasi nilai yang dapat dimasukkan dan menggantikan tabel lookup untuk data yang bersifat fixed-value.

4. Implementasi PostgreSQL

4.1 Lingkungan dan Versi

Komponen	Versi / Keterangan
PostgreSQL	v16.x
Python	3.11+
Streamlit	≥ 1.32
psycopg2	≥ 2.9
OS	Windows / Linux
Tools	psql, pgAdmin / DBeaver, VS Code

4.2 Pembuatan Database dan Tipe Data ENUM

```
-- Membuat database
CREATE DATABASE manajemen_kos;

-- Menghubungkan ke database
\c manajemen_kos;

-- Membuat tipe data ENUM
CREATE TYPE status_penyewa AS ENUM ('Aktif', 'Tidak aktif');
CREATE TYPE status_kamar AS ENUM ('Sedang disewa', 'Kosong');
CREATE TYPE status_maintenance AS ENUM ('Tertunda', 'Sedang dikerjakan',
'Selesai');
CREATE TYPE metode_pembayaran AS ENUM ('Tunai', 'Transfer');
CREATE TYPE status_pembayaran AS ENUM ('Gagal', 'Berhasil');
```

Tipe ENUM digunakan untuk membatasi nilai yang dapat dimasukkan ke dalam kolom tertentu, menggantikan tabel referensi sederhana dan memastikan integritas data secara native di level PostgreSQL.

4.3 Definisi Tabel (DDL)

```
-- =====
-- TABEL: roles
-- Deskripsi: Menyimpan peran pengguna dalam sistem
-- =====
CREATE TABLE IF NOT EXISTS roles (
    id_roles SERIAL PRIMARY KEY,
    nama_role VARCHAR(25) NOT NULL UNIQUE
);

-- =====
-- TABEL: users
-- Deskripsi: Akun pengguna untuk autentikasi sistem
-- =====
CREATE TABLE IF NOT EXISTS users (
    id_user SERIAL PRIMARY KEY,
    email VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    id_roles INT NOT NULL,
    FOREIGN KEY (id_roles) REFERENCES roles(id_roles)
);

-- =====
-- TABEL: profil_penyewa
-- Deskripsi: Data identitas lengkap penyewa
-- =====
CREATE TABLE IF NOT EXISTS profil_penyewa (
    id_penyewa SERIAL PRIMARY KEY,
    nama_lengkap VARCHAR(100) NOT NULL,
    pekerjaan VARCHAR(50) NOT NULL,
    instansi VARCHAR(50) NOT NULL,
    deskripsi_pekerjaan VARCHAR(100) NOT NULL,
    status status_penyewa NOT NULL DEFAULT 'Tidak aktif',
    url_ktp VARCHAR(255) NOT NULL,
    id_user INT NOT NULL UNIQUE,
    FOREIGN KEY (id_user) REFERENCES users(id_user)
);

-- =====
-- TABEL: kos
-- Deskripsi: Data properti kos yang dikelola sistem
-- =====
CREATE TABLE IF NOT EXISTS kos (
    id_kos SERIAL PRIMARY KEY,
    nama_kos VARCHAR(50) NOT NULL UNIQUE,
    alamat TEXT NOT NULL
);
```

```

-- =====
-- TABEL: tipe_kamar
-- Deskripsi: Kategori kamar dengan fasilitas dan harga sewa
-- =====
CREATE TABLE IF NOT EXISTS tipe_kamar (
    id_tipe_kamar      SERIAL PRIMARY KEY,
    kategori           VARCHAR(50) NOT NULL UNIQUE,
    deskripsi_fasilitas VARCHAR(255) NOT NULL,
    harga_sewa         INT NOT NULL
);

-- =====
-- TABEL: kamar
-- Deskripsi: Data setiap unit kamar pada suatu kos
-- =====
CREATE TABLE IF NOT EXISTS kamar (
    id_kamar           SERIAL PRIMARY KEY,
    nomor              VARCHAR(8) NOT NULL,
    luas               INT NOT NULL,
    lantai             VARCHAR(2) NOT NULL,
    status             status_kamar NOT NULL DEFAULT 'Kosong',
    id_tipe_kamar      INT NOT NULL,
    id_kos              INT NOT NULL,
    FOREIGN KEY (id_tipe_kamar) REFERENCES tipe_kamar(id_tipe_kamar),
    FOREIGN KEY (id_kos) REFERENCES kos(id_kos)
);

-- =====
-- TABEL: sewa
-- Deskripsi: Transaksi sewa kamar oleh penyewa
-- =====
CREATE TABLE IF NOT EXISTS sewa (
    id_sewa            SERIAL PRIMARY KEY,
    tanggal_mulai      DATE NOT NULL,
    tanggal_akhir      DATE NOT NULL,
    id_penyewa         INT NOT NULL,
    id_kamar           INT NOT NULL,
    FOREIGN KEY (id_penyewa) REFERENCES profil_penyewa(id_penyewa),
    FOREIGN KEY (id_kamar) REFERENCES kamar(id_kamar)
);

-- =====
-- TABEL: pembayaran
-- Deskripsi: Data pembayaran dari suatu transaksi sewa
-- =====
CREATE TABLE IF NOT EXISTS pembayaran (
    id_pembayaran      SERIAL PRIMARY KEY,
    nominal            INT NOT NULL,
    metode_bayar       metode_pembayaran NOT NULL,
    id_sewa            INT NOT NULL,
    FOREIGN KEY (id_sewa) REFERENCES sewa(id_sewa)
);

-- =====

```



```
-- TABEL: periode_pembayaran
-- Deskripsi: Rincian periode bulanan dari suatu pembayaran
-- =====
CREATE TABLE IF NOT EXISTS periode_pembayaran (
    id_periode      SERIAL PRIMARY KEY,
    periode_bayar  VARCHAR(10) NOT NULL,
    status          status_pembayaran NOT NULL,
    id_pembayaran  INT NOT NULL,
    FOREIGN KEY (id_pembayaran) REFERENCES pembayaran(id_pembayaran)
);

-- =====
-- TABEL: request_maintenance
-- Deskripsi: Pengajuan permintaan perbaikan oleh penyewa
-- =====
CREATE TABLE IF NOT EXISTS request_maintenance (
    id_request_maintenance SERIAL PRIMARY KEY,
    deskripsi              VARCHAR(255) NOT NULL,
    id_penyewa              INT NOT NULL,
    FOREIGN KEY (id_penyewa) REFERENCES profil_penyewa(id_penyewa)
);

-- =====
-- TABEL: riwayat_maintenance
-- Deskripsi: Riwayat penanganan permintaan maintenance
-- =====
CREATE TABLE IF NOT EXISTS riwayat_maintenance (
    id_riwayat_maintenance SERIAL PRIMARY KEY,
    tanggal_mulai           DATE,
    tanggal_selesai         DATE,
    status                  status_maintenance NOT NULL DEFAULT 'Tertunda',
    id_request_maintenance INT NOT NULL UNIQUE,
    FOREIGN KEY (id_request_maintenance) REFERENCES
request_maintenance(id_request_maintenance)
);
```

4.4 Implementasi Relasi

Seluruh relasi antar tabel diimplementasikan menggunakan Foreign Key constraint yang didefinisikan langsung saat **CREATE TABLE**. Berikut ringkasan implementasi relasi:

No	Tabel Anak	Kolom FK	Tabel Induk	Jenis Relasi
1	users	id_roles	roles	1:N
2	profil_penyewa	id_user	users	1:1 (opsional)
3	kamar	id_kos	kos	1:N
4	kamar	id_tipe_kamar	tipe_kamar	1:N
5	sewa	id_penyewa	profil_penyewa	1:N (opsional)

No	Tabel Anak	Kolom FK	Tabel Induk	Jenis Relasi
6	sewa	id_kamar	kamar	1:N (opsional)
7	pembayaran	id_sewa	sewa	1:N
8	periode_pembayaran	id_pembayaran	pembayaran	1:N
9	request_maintenance	id_penyewa	profil_penyewa	1:N (opsional)
10	riwayat_maintenance	id_request_maintenance	request_maintenance	1:1 (opsional)

4.5 Generate Data Dummy (DML)

Data dummy digenerate menggunakan fungsi `generate_series()` dan `random()` PostgreSQL untuk menghasilkan data realistis dalam jumlah besar. Berikut jumlah data yang berhasil digenerate:

Tabel	Jumlah Data	Keterangan
roles	2	Admin, Penyewa
kos	2	Hidden Kost Origin, Hidden Kost Lux
tipe_kamar	3	Tipe A (Rp 850.000), Tipe B (Rp 1.000.000), Tipe C (Rp 1.500.000)
users	1.000	2 admin + 998 penyewa
profil_penyewa	450	User berperan sebagai penyewa
kamar	300	150 per kos, 3 lantai, 3 tipe kamar
sewa	>50.000	Objek utama pengujian performa
pembayaran	>50.000	1 pembayaran per sewa
periode_pembayaran	>50.000	1 periode per pembayaran
request_maintenance	~180	~40% dari penyewa
riwayat_maintenance	~180	1 riwayat per request

Contoh script generate data untuk tabel sewa (tabel utama pengujian):

```
-- Generate >50.000 transaksi sewa untuk pengujian performa
INSERT INTO sewa (tanggal_mulai, tanggal_akhir, id_penyewa, id_kamar)
SELECT
    start_date,
    start_date + (ARRAY[30,90,180,365,730])[floor(random()*5+1)] * INTERVAL '1
day',
    p.id_penyewa,
    k.id_kamar
FROM (
    SELECT CURRENT_DATE - ((random()*1500)::int * INTERVAL '1 day') AS start_date
```

```

    FROM generate_series(1, 50000)
  ) t
  CROSS JOIN LATERAL (
    SELECT id_penyewa FROM profil_penyewa ORDER BY random() LIMIT 1
  ) p
  CROSS JOIN LATERAL (
    SELECT id_kamar FROM kamar ORDER BY random() LIMIT 1
  ) k;

```

4.6 Query Operasional Utama (DML)

Query 1 — Daftar Kamar Beserta Status Sewa Aktif (Dashboard Pemilik)

```

-- Menampilkan semua kamar lengkap dengan penyewa aktif menggunakan LATERAL JOIN
SELECT k.nomor          AS "Nomor",
       ko.nama_kos      AS "Kos",
       k.lantai         AS "Lantai",
       tk.kategori      AS "Tipe",
       tk.harga_sewa    AS "Harga (Rp)",
       k.status::TEXT   AS "Status",
       COALESCE(pp.nama_lengkap, '-') AS "Penyewa",
       COALESCE(TO_CHAR(s_aktif.tanggal_mulai, 'DD-MM-YYYY'), '-') AS "Sewa
Mulai",
       COALESCE(TO_CHAR(s_aktif.tanggal_akhir, 'DD-MM-YYYY'), '-') AS "Sewa Akhir"
FROM kamar k
JOIN kos ko ON k.id_kos = ko.id_kos
JOIN tipe_kamar tk ON k.id_tipe_kamar = tk.id_tipe_kamar
LEFT JOIN LATERAL (
  SELECT s.id_penyewa, s.tanggal_mulai, s.tanggal_akhir
  FROM sewa s
  WHERE s.id_kamar = k.id_kamar
        AND s.tanggal_akhir >= CURRENT_DATE
        ORDER BY s.tanggal_mulai DESC LIMIT 1
) s_aktif ON TRUE
LEFT JOIN profil_penyewa pp ON s_aktif.id_penyewa = pp.id_penyewa
ORDER BY ko.nama_kos, k.nomor;

```

Query 2 — Riwayat Sewa Aktif per Penyewa (Dashboard Penyewa)

```

-- Menampilkan sewa aktif penyewa tertentu beserta informasi jatuh tempo
SELECT s.id_sewa,
       k.nomor AS "Kamar",
       ko.nama_kos AS "Kos",
       tk.harga_sewa AS harga,
       s.tanggal_mulai,
       s.tanggal_akhir,
       (s.tanggal_akhir - CURRENT_DATE) AS "Sisa Hari",
       (s.tanggal_mulai + (COALESCE(bayar.jumlah_bulan_bayar, 0)

```

```
        * INTERVAL '1 month'))::date AS jatuh_tempo
FROM sewa s
JOIN kamar k ON s.id_kamar = k.id_kamar
JOIN kos ko ON k.id_kos = ko.id_kos
JOIN tipe_kamar tk ON k.id_tipe_kamar = tk.id_tipe_kamar
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS jumlah_bulan_bayar
    FROM pembayaran pb
    JOIN periode_pembayaran per ON pb.id_pembayaran = per.id_pembayaran
    WHERE pb.id_sewa = s.id_sewa AND per.status = 'Berhasil'
) bayar ON TRUE
WHERE s.id_penyewa = %s
AND s.tanggal_akhir >= CURRENT_DATE
ORDER BY s.tanggal_mulai DESC;
```

Query 3 — Riwayat Pembayaran Aktif (Tab Pembayaran)

```
-- Menampilkan riwayat pembayaran dari sewa yang masih aktif
SELECT pp.nama_lengkap AS "Penyewa",
       k.nomor AS "Kamar",
       ko.nama_kos AS "Kos",
       pb.nominal AS "Nominal (Rp)",
       pb.metode_bayar::TEXT AS "Metode",
       per.periode_bayar AS "Periode",
       per.status::TEXT AS "Status Bayar"
FROM pembayaran pb
JOIN sewa s ON pb.id_sewa = s.id_sewa
JOIN profil_penyewa pp ON s.id_penyewa = pp.id_penyewa
JOIN kamar k ON s.id_kamar = k.id_kamar
JOIN kos ko ON k.id_kos = ko.id_kos
JOIN periode_pembayaran per ON pb.id_pembayaran = per.id_pembayaran
WHERE s.tanggal_akhir >= CURRENT_DATE
ORDER BY per.periode_bayar DESC
LIMIT 500;
```

5. Strategi Penggunaan Indeks

5.1 Analisis Pola Akses Data

Sebelum membuat indeks, dilakukan analisis terhadap pola query aktual dari kode Streamlit ([halaman.py](#)) untuk mengidentifikasi kolom yang paling sering digunakan pada klausa `WHERE` dan `JOIN`:

Kolom	Tabel	Digunakan pada	Frekuensi	Kandidat Indeks
id_penyewa	sewa	WHERE, JOIN	Sangat Tinggi	Ya

Kolom	Tabel	Digunakan pada	Frekuensi	Kandidat Indeks
id_kamar	sewa	WHERE, JOIN (LATERAL)	Sangat Tinggi	Ya
tanggal_akhir	sewa	WHERE (filter aktif)	Sangat Tinggi	Ya
nama_lengkap	profil penyewa	WHERE (pencarian)	Tinggi	Ya
status	profil penyewa	WHERE (filter)	Tinggi	Ya (low selectivity)
status	kamar	WHERE (filter)	Tinggi	Ya (low selectivity)
nomor	kamar	WHERE (lookup)	Sedang	Ya
id_sewa	pembayaran	JOIN	Tinggi	Ya
id_pembayaran	periode pembayaran	JOIN	Tinggi	Ya
id penyewa	request maintenance	WHERE	Sedang	Ya

5.2 Pembuatan Indeks

Indeks dari DDL (Didefinisikan Saat Pembuatan Tabel)

```
-- =====
-- INDEX: index_nama_penyewa
-- Tujuan: Mempercepat pencarian penyewa berdasarkan nama (exact match)
-- Tipe: B-Tree (default)
-- =====
CREATE INDEX index_nama_penyewa ON profil_penyewa (nama_lengkap);

-- =====
-- INDEX: index_nomor_kamar
-- Tujuan: Mempercepat lookup kamar berdasarkan nomor
-- Tipe: B-Tree
-- =====
CREATE INDEX index_nomor_kamar ON kamar (nomor);

-- =====
-- INDEX: index_status_penyewa
-- Tujuan: Mempercepat filter penyewa aktif/tidak aktif
-- Tipe: B-Tree (efektif untuk Bitmap Index Scan)
-- =====
CREATE INDEX index_status_penyewa ON profil_penyewa (status);

-- =====
-- INDEX: index_status_kamar
-- Tujuan: Mempercepat filter kamar kosong/sedang disewa
-- Tipe: B-Tree (efektif untuk Bitmap Index Scan)
```

```
-- =====
CREATE INDEX index_status_kamar ON kamar (status);
```

Indeks Tambahan (Dibuat Dinamis via Halaman Optimasi Query)

```
-- =====
-- INDEX: index_id_penyewa_sewa
-- Tujuan: Mempercepat lookup riwayat sewa per penyewa (FK join)
-- Dipakai di: Dashboard Penyewa → tab Status Sewa
-- =====
CREATE INDEX index_id_penyewa_sewa ON sewa (id_penyewa);

-- =====
-- INDEX: index_id_kamar_sewa
-- Tujuan: Mempercepat lookup riwayat sewa per kamar (LATERAL JOIN)
-- Dipakai di: Dashboard Pemilik → tab Data Kamar
-- =====
CREATE INDEX index_id_kamar_sewa ON sewa (id_kamar);
```

5.3 Justifikasi Pemilihan Indeks

Nama Indeks	Tipe	Alasan Pembuatan
index_nama_penyewa	B-Tree	Kolom nama_lengkap sering digunakan untuk pencarian exact match; B-Tree mendukung = operator secara efisien
index_nomor_kamar	B-Tree	Nilai nomor hampir unik per kamar (KO101, KL205, dll), sehingga Index Scan sangat efektif
index_status_penyewa	B-Tree	Filter status = 'Aktif' digunakan hampir di setiap halaman; PostgreSQL akan memilih Bitmap Index Scan untuk low-cardinality ENUM
index_status_kamar	B-Tree	Sama dengan di atas; filter kamar kosong/terisi adalah query paling sering di dashboard
index_id_penyewa_sewa	B-Tree	FK lookup bersifat highly selective (satu penyewa = sebagian kecil sewa); Index Scan jauh lebih efisien dari Seq Scan pada 50.000+ baris
index_id_kamar_sewa	B-Tree	Sama dengan di atas; LATERAL JOIN pada id_kamar dieksekusi untuk setiap baris kamar (300 kamar × 1 LATERAL = 300 eksekusi)

5.4 Verifikasi Indeks yang Dibuat

```
-- Melihat semua indeks pada database
SELECT
  tablename AS "Tabel",
  indexname AS "Nama Index",
  indexdef AS "Definisi"
```

```
FROM pg_indexes
WHERE schemaname = 'public'
ORDER BY tablename, indexname;
```

6. Analisis Query dengan EXPLAIN

6.1 Metodologi Pengujian

Pengujian performa dilakukan menggunakan perintah **EXPLAIN ANALYZE** yang memberikan execution plan aktual (bukan estimasi) dari PostgreSQL, termasuk jenis scan, estimasi cost, jumlah baris yang diproses, dan waktu eksekusi nyata.

Pendekatan pengujian:

- Setiap query utama diuji dalam **dua kondisi**: tanpa index (memaksa Seq Scan dengan **DROP INDEX**) dan dengan index (setelah **CREATE INDEX + ANALYZE**).
- Perintah **ANALYZE** dijalankan setelah setiap perubahan index agar statistik tabel selalu up-to-date.
- Data dummy yang digunakan: **>50.000 baris** pada tabel **sewa**, 450 baris pada **profil_penyewa**, 300 baris pada **kamar**.
- Pengujian dilakukan melalui halaman **Optimasi Query** pada dashboard Streamlit.

```
-- Template perintah EXPLAIN yang digunakan
EXPLAIN ANALYZE
SELECT * FROM sewa WHERE id_penyewa = 1;
```

6.2 Analisis Query 1 — Riwayat Sewa per Penyewa

Query:

```
SELECT * FROM sewa WHERE id_penyewa = 1;
```

Konteks penggunaan: Query ini dieksekusi di Dashboard Penyewa (tab Status Sewa) untuk menampilkan seluruh sewa aktif milik penyewa yang sedang login.

Sebelum Indeks (Seq Scan)

```
Seq Scan on sewa (cost=0.00..959.60 rows=2 width=20)
    (actual time=2.693..2.693 rows=0.00 loops=1)
  Filter: (id_penyewa = 1)
  Rows Removed by Filter: 50768
  Buffers: shared hit=325
Planning:
  Buffers: shared hit=17
```

```
Planning Time: 0.106 ms
Execution Time: 2.704 ms
```

Analisis Sebelum Indeks:

Metrik	Nilai
Metode Scan	Sequential Scan
Estimated Cost	0.00..959.60
Actual Execution Time	2.704 ms
Rows Processed	50.768 baris (seluruh tabel)
Rows Matched	0 (penyewa id=1 tidak punya sewa)
Rows Removed by Filter	50.768 baris

PostgreSQL membaca **seluruh 50.768 baris** tabel sewa hanya untuk menemukan baris yang `id_penyewa = 1`. Karena tidak ada index, tidak ada cara lain selain memeriksa setiap baris satu per satu.

Sesudah Indeks (Index Scan)

```
Index Scan using index_id_penyewa_sewa on sewa
(cost=0.29..8.53 rows=2 width=20)
(actual time=0.022..0.023 rows=0.00 loops=1)
Index Cond: (id_penyewa = 1)
Index Searches: 1
Buffers: shared read=2
Planning:
  Buffers: shared hit=17 read=1
Planning Time: 0.170 ms
Execution Time: 0.032 ms
```

Analisis Sesudah Indeks:

Metrik	Nilai
Metode Scan	Index Scan
Index Digunakan	<code>index_id_penyewa_sewa</code>
Estimated Cost	0.29..8.53
Actual Execution Time	0.032 ms
Buffers	2 page (vs 325 sebelumnya)

Perbandingan Performa Query 1

Metrik	Sebelum Indeks	Sesudah Indeks	Peningkatan
Execution Time	2.704 ms	0.032 ms	~84× lebih cepat
Scan Method	Seq Scan	Index Scan	—
Baris Dibaca	50.768	2 halaman	Drastis berkurang
Buffer Hit	325 pages	2 pages	—

Kesimpulan Q1: Index `index_id_penyewa_sewa` memberikan peningkatan performa yang sangat signifikan. Execution time turun dari 2.704 ms menjadi 0.032 ms, setara dengan peningkatan ~84×. Ini membuktikan bahwa index pada kolom FK yang sering difilter sangat krusial pada tabel besar.

🔗 **[ARAHAN]:** Sisipkan screenshot output Streamlit untuk skenario 1 (Tanpa Index vs Dengan Index) di sini.

6.3 Analisis Query 2 — Pencarian Nama Penyewa (Exact vs LIKE)

Query 2a — Exact Match:

```
SELECT * FROM profil_penyewa WHERE nama_lengkap = 'Penyewa ke-1';
```

Query 2b — LIKE Wildcard:

```
SELECT * FROM profil_penyewa WHERE nama_lengkap LIKE '%Penyew%';
```

Konteks penggunaan: Query ini digunakan untuk fitur pencarian penyewa berdasarkan nama pada tab Data Penyewa di dashboard pemilik.

Query 2a — Exact Match (Index Scan)

```
Index Scan using index_nama_penyewa on profil_penyewa
(cost=0.27..8.29 rows=1 width=82)
(actual time=0.039..0.050 rows=1.00 loops=1)
Index Cond: ((nama_lengkap)::text = 'Penyewa ke-1'::text)
Index Searches: 1
Buffers: shared hit=3
Planning Time: 0.097 ms
Execution Time: 0.065 ms
```

Analisis Query 2a:

Metrik	Nilai
Metode Scan	Index Scan

Metrik	Nilai
Index Digunakan	<code>index_nama_penyewa</code>
Execution Time	0.065 ms
Rows Matched	1 baris
Buffer Hit	3 pages

Query 2b — LIKE Wildcard (Seq Scan — Index Tidak Terpakai)

```
Seq Scan on profil_penyewa (cost=0.00..12.62 rows=450 width=82)
  (actual time=0.013..0.084 rows=450.00 loops=1)
  Filter: ((nama_lengkap)::text ~~ '%Penyew%')::text)
  Buffers: shared hit=7
  Planning Time: 0.077 ms
  Execution Time: 0.107 ms
```

Analisis Query 2b:

Metrik	Nilai
Metode Scan	Sequential Scan
Index Digunakan	— (tidak ada)
Execution Time	0.107 ms
Rows Matched	450 baris (seluruh tabel)

Perbandingan Performa Query 2

Metrik	Q2a (Exact Match)	Q2b (LIKE Wildcard)	Selisih
Execution Time	0.065 ms	0.107 ms	Q2a ~1.6× lebih cepat
Scan Method	Index Scan	Seq Scan	—
Index Digunakan	<code>index_nama_penyewa</code>	Tidak ada	—
Rows Dibaca	1 baris	450 baris	—

Kesimpulan Q2: Index B-tree `index_nama_penyewa` hanya efektif untuk pencarian exact match (operator `=`). Ketika wildcard `%` diletakkan di awal pattern (`LIKE '%...%'`), PostgreSQL tidak dapat menggunakan B-tree index karena B-tree hanya mendukung *prefix search*. Akibatnya, query LIKE selalu menghasilkan Seq Scan. Untuk mendukung full-text search yang efisien, diperlukan index GIN dengan ekstensi `pg_trgm`.

 **[ARAHAN]:** Sisipkan screenshot output Streamlit untuk skenario 2 (Exact vs LIKE) di sini.

6.4 Analisis Query 3 — Riwayat Sewa per Kamar

Query:

```
SELECT * FROM sewa WHERE id_kamar = 1;
```

Konteks penggunaan: Query ini dieksekusi via LATERAL JOIN di tab Data Kamar (Dashboard Pemilik) untuk setiap baris kamar yang ditampilkan — artinya dieksekusi hingga 300 kali per halaman load.

Sebelum Indeks (Seq Scan)

```
Seq Scan on sewa (cost=0.00..959.60 rows=2 width=20)
  (actual time=0.046..3.023 rows=1.00 loops=1)
  Filter: (id_kamar = 1)
  Rows Removed by Filter: 50767
  Buffers: shared hit=325
Planning:
  Buffers: shared hit=17
Planning Time: 0.113 ms
Execution Time: 3.034 ms
```

Analisis Sebelum Indeks:

Metrik	Nilai
Metode Scan	Sequential Scan
Estimated Cost	0.00..959.60
Actual Execution Time	3.034 ms
Rows Processed	50.768 baris (seluruh tabel)
Rows Removed by Filter	50.767 baris

Sesudah Indeks (Index Scan)

```
Index Scan using index_id_kamar_sewa on sewa
  (cost=0.29..8.43 rows=2 width=20)
  (actual time=0.024..0.024 rows=1.00 loops=1)
  Index Cond: (id_kamar = 1)
  Index Searches: 1
  Buffers: shared hit=1 read=2
Planning:
  Buffers: shared hit=17 read=1
Planning Time: 0.126 ms
Execution Time: 0.033 ms
```

Analisis Sesudah Indeks:

Metrik	Nilai
Metode Scan	Index Scan
Index Digunakan	<code>index_id_kamar_sewa</code>
Estimated Cost	0.29..8.43
Actual Execution Time	0.033 ms
Buffers	3 pages (vs 325 sebelumnya)

Perbandingan Performa Query 3

Metrik	Sebelum Indeks	Sesudah Indeks	Peningkatan
Execution Time	3.034 ms	0.033 ms	~92× lebih cepat
Scan Method	Seq Scan	Index Scan	—
Baris Dibaca	50.768	3 pages	Drastis berkurang
Dampak LATERAL	~910 detik total (300 kamar × 3 ms)	~10 detik total (300 × 0.033 ms)	Kritis untuk UX

Kesimpulan Q3: Query ini memiliki dampak performa paling kritis karena dieksekusi 300 kali (satu per kamar) melalui LATERAL JOIN. Tanpa index, total waktu load tab Data Kamar bisa mencapai ~900 ms hanya untuk query ini. Dengan index `index_id_kamar_sewa`, total turun menjadi ~10 ms. Peningkatan ~92× menjadikan index ini prioritas tertinggi dalam sistem.

 **[ARAHAN]:** Sisipkan screenshot output Streamlit untuk skenario 3 (Tanpa Index vs Dengan Index) di sini.

6.5 Ringkasan Hasil Analisis EXPLAIN

Query	Sebelum (ms)	Sesudah (ms)	Scan Sebelum	Scan Sesudah	Speedup
Q1 — Riwayat Sewa per Penyewa	2.704	0.032	Seq Scan	Index Scan	~84×
Q2a — Cari Nama (Exact Match)	—	0.065	—	Index Scan	Baseline index
Q2b — Cari Nama (LIKE Wildcard)	0.107	0.107	Seq Scan	Seq Scan	Tidak berubah
Q3 — Riwayat Sewa per Kamar	3.034	0.033	Seq Scan	Index Scan	~92×

6.6 Temuan dan Diskusi

1. Kapan PostgreSQL memilih Seq Scan meskipun index tersedia?

Pada Query 2b (LIKE wildcard), meskipun `index_nama_penyewa` tersedia, PostgreSQL tetap memilih Seq Scan. Ini terjadi karena index B-tree tidak dapat digunakan untuk pola `LIKE '%...%'` — wildcard di awal string mencegah binary search. B-tree hanya mendukung *prefix search* (`LIKE 'Penyewa%'`).

Selain itu, pada tabel kecil (`profil_penyewa` hanya 450 baris), PostgreSQL kadang memilih Seq Scan meskipun index tersedia, karena overhead membaca index lalu heap page justru lebih mahal dari langsung scan tabel kecil.

2. Index Scan vs Bitmap Index Scan

Untuk kolom dengan selectivity tinggi (`id_penyewa`, `id_kamar`), PostgreSQL memilih **Index Scan** karena hanya sedikit baris yang cocok — lebih efisien langsung melompat ke baris target. Untuk kolom ENUM dengan selectivity rendah (`status_kamar`, `status_penyewa` — hanya 2 nilai), PostgreSQL cenderung memilih **Bitmap Index Scan**: mengumpulkan semua pointer heap page terlebih dahulu, lalu membaca disk secara terurut, yang lebih efisien ketika banyak baris cocok.

3. Dampak ANALYZE terhadap planner

Setelah membuat index baru, perintah `ANALYZE` wajib dijalankan agar PostgreSQL memperbarui statistik tabel. Tanpa `ANALYZE`, planner mungkin masih memilih Seq Scan karena statistiknya belum mencerminkan keberadaan index baru.

4. Trade-off index: baca vs tulis

Index mempercepat operasi SELECT secara signifikan, namun menambah overhead pada INSERT/UPDATE/DELETE karena setiap perubahan data juga harus memperbarui struktur index. Untuk sistem kos yang lebih banyak membaca (query dashboard) dibanding menulis (transaksi baru), trade-off ini sangat menguntungkan.

7. Kesimpulan

7.1 Kesimpulan

1. Skema ERD Sistem Informasi Hidden Kos berhasil dirancang dengan **11 entitas** dan **10 relasi** yang memenuhi bentuk normal 3NF, merepresentasikan seluruh kebutuhan data operasional manajemen kos-kosan.
2. Implementasi pada PostgreSQL berhasil dilakukan dengan 11 tabel, 5 tipe ENUM, dan data dummy berskala besar — tabel `sewa` mencapai **>50.000 baris**, yang cukup representatif untuk menunjukkan perbedaan performa antara Seq Scan dan Index Scan.
3. Penerapan **6 index B-Tree** (4 dari DDL + 2 dinamis) terbukti mampu meningkatkan performa query secara dramatis: Query 1 (riwayat sewa per penyewa) meningkat **~84x** dan Query 3 (riwayat sewa per kamar) meningkat **~92x**.

4. Analisis **EXPLAIN ANALYZE** menunjukkan bahwa index paling efektif digunakan pada kolom FK dengan selectivity tinggi (**id_penyewa**, **id_kamar** di tabel **sewa**). Sebaliknya, index B-tree tidak efektif untuk pola **LIKE '%...%'** karena keterbatasan struktural B-tree.
5. PostgreSQL terbukti mampu memilih jenis scan yang optimal secara otomatis (Index Scan vs Bitmap Index Scan vs Seq Scan) berdasarkan statistik data aktual, tanpa perlu konfigurasi manual dari developer.

7.2 Saran dan Rekomendasi

- Tambahkan **index GIN dengan ekstensi pg_trgm** pada kolom **nama_lengkap** untuk mendukung pencarian substring (**LIKE '%...%'**) secara efisien tanpa Seq Scan.
- Pertimbangkan **partial index** pada tabel **sewa** untuk sewa aktif saja: **CREATE INDEX idx_sewa_aktif ON sewa(id_penyewa) WHERE tanggal_akhir >= CURRENT_DATE;** — mengurangi ukuran index dan meningkatkan performa query sewa aktif.
- Jadwalkan **VACUUM ANALYZE** secara berkala (atau aktifkan autovacuum) agar statistik tabel selalu up-to-date sehingga planner PostgreSQL dapat membuat keputusan yang akurat.
- Monitor performa index secara berkelanjutan menggunakan **pg_stat_user_indexes** untuk mengidentifikasi index yang tidak pernah digunakan dan dapat dihapus untuk menghemat storage.
- Untuk pengembangan lebih lanjut, implementasikan **composite index** pada **sewa(id_penyewa, tanggal_akhir)** untuk mengoptimalkan query sewa aktif per penyewa yang merupakan query paling sering dieksekusi di sistem.

8. Referensi

1. PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation — Chapter 11: Indexes*. <https://www.postgresql.org/docs/16/indexes.html>
2. PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation — Using EXPLAIN*. <https://www.postgresql.org/docs/16/using-explain.html>
3. Momjian, B. (2001). *PostgreSQL: Introduction and Concepts*. Addison-Wesley.
4. Streamlit Inc. (2024). *Streamlit Documentation*. <https://docs.streamlit.io>
5. Ramakrishnan, R., & Gehrke, J. (2003). *Database Management Systems* (3rd ed.). McGraw-Hill.
6. Faroult, S., & Robson, P. (2006). *The Art of SQL*. O'Reilly Media.

9. Lampiran

Lampiran A — Script DDL Lengkap

 **[ARAHAN]: Tempelkan isi file Database/DDL.SQL di sini secara lengkap.**

```
-- [Lihat file Database/DDL.SQL pada repository]
```

Lampiran B — Script Pembuatan Indeks Lengkap

```
-- Index dari DDL (otomatis saat setup)
CREATE INDEX index_nama_penyewa    ON profil_penyewa (nama_lengkap);
CREATE INDEX index_nomor_kamar     ON kamar (nomor);
CREATE INDEX index_status_penyewa  ON profil_penyewa (status);
CREATE INDEX index_status_kamar    ON kamar (status);

-- Index tambahan (dibuat via halaman Optimasi Query)
CREATE INDEX index_id_penyewa_sewa ON sewa (id_penyewa);
CREATE INDEX index_id_kamar_sewa   ON sewa (id_kamar);
```

Lampiran C — Data Dummy Lengkap

 **[ARAHAN]: Tempelkan isi file Database/DML.SQL di sini secara lengkap.**

```
-- [Lihat file Database/DML.SQL pada repository]
```

Lampiran D — Screenshot Output EXPLAIN ANALYZE

 **[ARAHAN]: Masukkan screenshot berikut dari dashboard Streamlit:**

- Gambar D.1 — Skenario 1: Q1 Tanpa Index (Seq Scan, 2.704 ms) vs Dengan Index (Index Scan, 0.032 ms)
- Gambar D.2 — Skenario 2: Q2a Exact Match (Index Scan, 0.065 ms) vs Q2b LIKE Wildcard (Seq Scan, 0.107 ms)
- Gambar D.3 — Skenario 3: Q3 Tanpa Index (Seq Scan, 3.034 ms) vs Dengan Index (Index Scan, 0.033 ms)

Lampiran E — Screenshot Dashboard Streamlit

 **[ARAHAN]: Masukkan screenshot halaman-halaman berikut:**

- Gambar E.1 — Halaman Dashboard Pemilik (tab Data Kamar)
- Gambar E.2 — Halaman Dashboard Pemilik (tab Pembayaran)
- Gambar E.3 — Halaman Dashboard Penyewa (tab Status Sewa)
- Gambar E.4 — Halaman Optimasi Query (perbandingan otomatis)

Lampiran F — Biodata Anggota

Nama	NIM	Prodi / Angkatan	Email
Muchammad Maulana	10241034	Sistem Informasi / 2024	10241034@student.itk.ac.id
Patra Ananda	10241061	Sistem Informasi / 2024	10241061@student.itk.ac.id
Tika Mila Wahyuni	10241070	Sistem Informasi / 2024	10241070@student.itk.ac.id
Vanessa Marie Tandiarru	10251118	Sistem Informasi / 2024	10251118@student.itk.ac.id

Laporan ini dibuat sebagai pemenuhan tugas besar mata kuliah Administrasi Basis Data, Program Studi Sistem Informasi, Institut Teknologi Kalimantan, Balikpapan, 2026.